Spring Scheduler



UP ASI
Bureau E204

Plan du Cours

- **Introduction**
- **@Scheduled Annotation**
- **Fixed delay vs Fixed rate**
- **Cron expression**
- **Affichage des résultats avec @Slf4j (Lombok) sur la console**

Introduction

- La planification(scheduling) consiste à exécuter les tâches pendant une période de temps spécifique.
- Spring Boot Scheduling est une fonctionnalité pratique qui nous permet de planifier des tâches dans nos applications Spring Boot.
- Par exemple, si vous voulez que votre application exécute une tâche après un intervalle fixe ou en fonction d'un calendrier.

- **Le scheduler fait partie du module Core du framework Spring (Pas de dépendances à ajouter dans le pom.xml).**

@Scheduled Annotation

- Spring Boot utilise l'annotation **@Scheduled** pour la planification des tâches.
- Il faut respecter certaines règles lors de l'utilisation de cette annotation : Les méthodes doivent être sans paramètre.
- Pour activer la planification (scheduling), il faut ajouter l'annotation **@EnableScheduling** à la classe main.

```
@EnableScheduling
@SpringBootApplication
public class TpFoyerApplication {
    public static void main(String[] args) {
        SpringApplication.run(TpFoyerApplication.class, args);
    }
}
```

@Scheduled Annotation

- Spring Boot permet de créer facilement une tâche de planification(scheduling task).

➢ Il suffit d'annoter la méthode avec l'annotation **@Scheduled**.

- L'annotation **@Scheduled** définit la planification (par exemple, quand la méthode sera exécutée, etc.).

- Nous pouvons passer certains paramètres à l'annotation pour personnaliser le comportement.

Fixed Rate

- Pour planifier un déclenchement de méthode à une date fixe, nous pouvons utiliser le paramètre **fixedRate** dans l'annotation **@Scheduled.**
- Fixed Rate : permet à Spring d'exécuter la tâche à des intervalles périodiques, même si la dernière exécution est en cours.




```
@Scheduled(fixedRate = 60000)
public void fixedRateMethod() {
system.out.println("Method with fixed Rate");
}
```

Fixed Delay

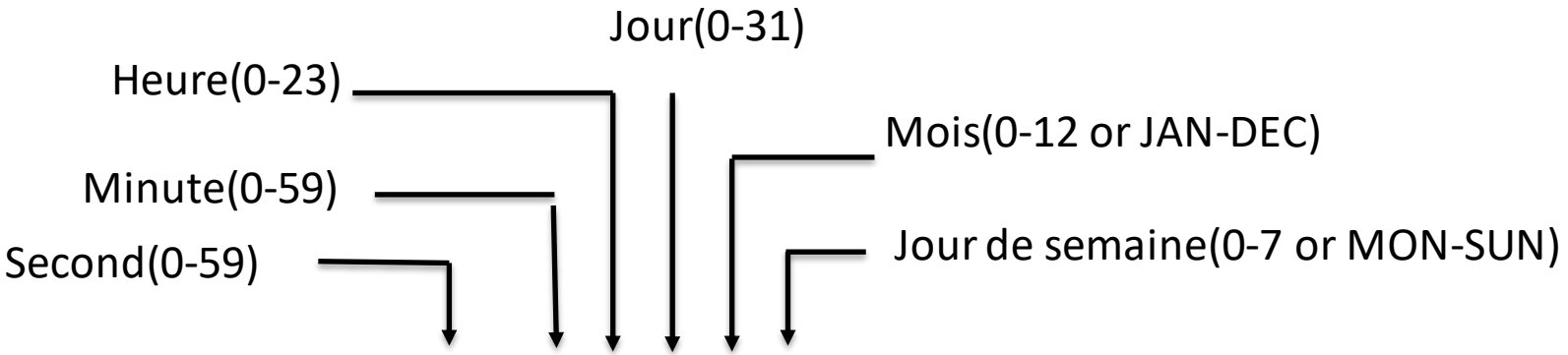
- Pour fixer un délai fixe entre la dernière exécution et le début de l'exécution suivante, nous pouvons utiliser le paramètre **fixedDelay**.
- Ce paramètre compte le délai après l'exécution de la dernière invocation.




```
@Scheduled(fixedDelay = 60000)
public void fixedDelayMethod() {
system.out.println("Method with fixed delay");
}
```

Cron expression

- L'expression **Cron** est une façon flexible et puissante pour planifier les tâches.




```
@Scheduled(cron = "*/60 * * * * *" )
public void cronMethod() {
system.out.println("Method with cron expression");
}
```

Exercice

1. `@Scheduled(cron = "15 * * * * *")`
2. `@Scheduled(cron = "0/15 * * * * *")`
3. `@Scheduled(cron = "0 0/30 11 * * *")`
4. `@Scheduled(cron = "0 0 8 ? 4 ?") or (cron = "0 0 8 * 4 *")`
5. `@Scheduled(cron = "0 0 9 14 2 SUN,TUE")`

Conclusion

field	Description
FixedRate	Nombre de millisecondes après l'heure de début de la dernière exécution de la méthode. => @retourne la période en millisecondes.
FixedDelay	Nombre de millisecondes après la dernière exécution de la méthode. ⇒ @retourne le délai en millisecondes.
cron	Écrire le cron et définir le calendrier. Vous pouvez également spécifier le fuseau horaire. => @retourne une expression qui peut être analysée dans un plan de cron

Affichage avec @Slf4j (Lombok) sur la console

Pour l'affichage des détails dans la console InteliiJ, nous utiliserons l'annotation **@Slf4j** (Lombok). Il suffit de :

- **1- Ajouter l'annotation @Slf4j au dessus du service concerné**
- **2- Ajouter, dans la méthode concernée, la ligne de code qui permet de faire l'affichage**

- (Voir détails page suivante) :

Affichage avec @Slf4j (Lombok) sur la console

```
@Service
@AllArgsConstructor
@Slf4j
public class ChambreServiceImpl implements IChambreService {

    ChambreRepository chambreRepository;

    @Scheduled(fixedDelay = 10000) // 10000 millisecondes
    public List<Chambre> retrieveAllChambres() {
        List<Chambre> listC = chambreRepository.findAll();
        for (Chambre c: listC) {
            log.info("Chambre :" + c);
        }
        return listC;
    }
```

Travail à faire

Spring Scheduler

1- Ajouter un Scheduler au dessus du service **public List<Bloc> retrieveAllBlocs()** pour lancer le service **toutes les minutes**.
2- Ajouter un message avec SLF4J pour afficher la liste des Blocs sur la console. (Voir exemple ci-dessus).

La planification : **@EnableScheduling** **@Scheduled**

L'affichage : **@Slf4j** ***log*.info** **@ToString**